

25 de septiembre de 2026.

N2-EXAM – 2026-20

Pregunta 1 (13 pts). Indique la **secuencia exacta** que imprime el siguiente programa de Python. **Encierre** en un círculo la letra de la única opción correcta. Obtendrá 13 pts al marcar la opción correcta. Si marca una opción incorrecta, no marca ninguna opción o marca más de una opción, la pregunta se le calificará con **0 pts**.

```
valores = {'principal': 1}
copia_valores = valores.copy()
valores['secundario'] = 2

def modificar_valores(datos: dict) -> None:
    print(datos)
    datos['principal'] = -777
    print(datos)

print(valores)
print(copia_valores)

resultado = modificar_valores(valores.copy())

print(valores)
print(copia_valores)
print("Vacio" if not resultado else resultado)
```

Solución:

```
{'principal': 1, 'secundario': 2}
{'principal': 1}
{'principal': 1, 'secundario': 2}
{'principal': -777, 'secundario': 2}
{'principal': 1, 'secundario': 2}
{'principal': 1}
Vacio
```

Las siguientes preguntas se basan en **N2-PROY: CupiAdmisiones**. Considere la siguiente información:

Recordatorio de la estructura de un candidato en N2-PROY:

Cada candidato se representa mediante un diccionario (`dict`) construido por la función `crear_candidato`, que contiene siempre estas 16 llaves, con los mismos nombres de los parámetros en esa función:

Llave	Tipo	Descripción
"nombre"	str	Nombre completo. Ejemplo: "Zutano Mengano".
"doc_identidad"	int	Documento de identidad. Siempre mayor que 0.
"edad"	int	Edad del candidato en años. Ejemplo: 18.
"genero"	str	Género del candidato. Ejemplo: "Masculino".
"fecha_postulacion"	str	Fecha de postulación. Ejemplo: "2026-06-21".
"nacionalidad"	str	Nacionalidad del candidato. Ejemplo: "Colombiana".
"num_premios_academicos"	int	Número de premios académicos obtenidos. Ejemplo: 3.
"puntaje_saber11"	int	Puntaje global obtenido en la Prueba Saber 11°. Ejemplo: 351.
"carrera_aplicada"	str	Carrera a la que aplica. Ejemplo: "Ingeniería Civil".
"facultad_aplicada"	str	Facultad de la carrera. Ejemplo: "Ingeniería".
"tiene_discapacidad"	bool	Indica si el candidato tiene alguna discapacidad.
"es_indigena"	bool	Indica si el candidato pertenece a una comunidad indígena.
"tiene_SISBEN"	bool	Indica si el candidato está registrado en el SISBEN.
"es_victima_conflicto"	bool	Indica si el candidato es víctima del conflicto armado.
"es_becado"	bool	Indica si el candidato ya cuenta con alguna beca.
"ingresos_nucleo_familiar"	float	Ingresos mensuales del núcleo familiar.

Recordatorio de la comparación de fechas en N2-PROY:

Todas las fechas (por ejemplo, "fecha_postulacion") siempre usan el formato "YYYY-MM-DD". En Python se comparan directamente como strings, porque el orden lexicográfico coincide con el cronológico:

Comparación	Resultado	Razón
"2005-02-15" < "2006-06-10"	True	2005 es anterior a 2006.
"2010-08-23" > "2009-12-31"	True	2010 es posterior a 2009.
"2015-03-10" < "2015-03-20"	True	Mismo año y mes, pero el día 10 es anterior al día 20.

Función de N2-PROY que puede invocar cuando la necesite:

Asuma que la siguiente función ya está implementada y que puede invocarla en este examen si la necesita.

```
puntaje_para_beca_gobierno(candidato: dict, max_ingreso_nucleo_familiar: float) -> float
```

La anterior función calcula el puntaje del candidato para aspirar a una beca del gobierno. Retorna `0.0` si el candidato no alcanza el puntaje Saber 11° mínimo de la facultad a la que aplica, o si ya es becado. Si no, suma `0.2` por cada una de estas cinco condiciones que cumpla: nacionalidad colombiana, al menos un premio académico, estar registrado en el SISBEN, ser víctima del conflicto armado, y tener ingresos del núcleo familiar menores o iguales a `max_ingreso_nucleo_familiar`. El resultado retornado va de `0.0` a `1.0`, redondeado a 2 decimales. Un resultado de `1.0` significa que cumple las cinco.

Recordatorio de aserciones:

En toda función de **N2-PROY** se validan sus parámetros con `assert`. Aquí un ejemplo de implementación de un `assert` en Python. Note que la aserción incluye un mensaje de error claro y conciso.

```
assert isinstance(edad, int), "Se esperaba una edad del tipo int, pero se recibió: " + str(type(edad))
```

Pregunta 2 (20 pts). Implemente la función `documento_repetido_en_pareja` en Python, que retorna el número de documento de identidad que comparten dos candidatos, si alguno.

- Las parejas de candidatos deben revisarse en este orden: $c1-c2$, $c1-c3$, $c1-c4$, $c2-c3$, $c2-c4$ y $c3-c4$.
- Si hay más de una pareja con el documento repetido, se retorna el documento de la **primera** pareja según ese orden.
- Asuma que los documentos de identidad de los cuatro candidatos, son siempre mayores que 0. Si los cuatro documentos son distintos, la función debe retornar 0.
- **Debe** usar condicionales y una variable de retorno. **No** puede usar `len()`, `min()` ni `max()`.

```
def documento_repetido_en_pareja(c1: dict, c2: dict, c3: dict, c4: dict) -> int:
    """
    Retorna el documento de identidad que comparte una pareja de candidatos, si alguno.

    Las parejas de candidatos deben revisarse en este orden:
    c 1 c2 , c 1 c3 , c 1 c4 , c 2 c3 , c 2 c4 y c 3 c4

    Args:
        c1, c2, c3, c4 (dict): Diccionarios que representan a los cuatro candidatos.

    Returns:
        int: Documento repetido (el primero en caso de empate), o 0 si los cuatro documentos son distintos.
    """
    repetido = 0

    if c1["doc_identidad"] == c2["doc_identidad"]:
        repetido = c1["doc_identidad"]
    elif c1["doc_identidad"] == c3["doc_identidad"]:
        repetido = c1["doc_identidad"]
    elif c1["doc_identidad"] == c4["doc_identidad"]:
        repetido = c1["doc_identidad"]
    elif c2["doc_identidad"] == c3["doc_identidad"]:
        repetido = c2["doc_identidad"]
    elif c2["doc_identidad"] == c4["doc_identidad"]:
        repetido = c2["doc_identidad"]
    elif c3["doc_identidad"] == c4["doc_identidad"]:
        repetido = c3["doc_identidad"]

    return repetido
```

Pregunta 3 (30 pts). Implemente la función `candidatos_en_periodo` en Python, que filtra y retorna los documentos de los candidatos cuya fecha de postulación esté dentro del periodo comprendido entre `fecha_inicial` y `fecha_final`. Note que esta función recibe **tres** candidatos y no cuatro. Esto es intencional, para reducirle la cantidad de código que debe escribir a mano durante el examen.

- Las dos fechas del periodo son inclusivas: un candidato que se postuló exactamente el día de apertura, o exactamente el día de cierre, queda dentro del periodo. Asuma que `fecha_inicial` nunca es posterior a `fecha_final`.
- Se retorna un string con los documentos de identidad de los candidatos filtrados, en el orden `c1` a `c3`, separados por una coma y un espacio. Por ejemplo, si los documentos de `c1` a `c3` son 1001, 1002 y 1003, y solo `c2` y `c3` quedan dentro del periodo, se retornaría: "1002, 1003".
- El string **no** debe empezar ni terminar con el separador, ni contener dos o más separadores seguidos. Esto importa cuando los candidatos filtrados no son consecutivos: en el ejemplo anterior, si solo `c1` y `c3` quedan dentro del periodo, se retornaría: "1001, 1003".
- Si ningún candidato se postuló dentro del periodo, se retorna "Ninguno".
- **Debe** usar condicionales y una variable de retorno. **No** puede usar `len()`, `min()` ni `max()`; **sí** puede usar `str()`.

```
def candidatos_en_periodo(fecha_inicial: str, fecha_final: str, c1: dict, c2: dict, c3: dict) -> str:
    """
    Filtra los candidatos que se postularon dentro del periodo dado y retorna sus documentos de identidad.
    """
    resultado = ""

    if fecha_inicial <= c1["fecha_postulacion"] <= fecha_final:
        if resultado != "":
            resultado += ", "
        resultado += str(c1["doc_identidad"])

    if fecha_inicial <= c2["fecha_postulacion"] <= fecha_final:
        if resultado != "":
            resultado += ", "
        resultado += str(c2["doc_identidad"])

    if fecha_inicial <= c3["fecha_postulacion"] <= fecha_final:
        if resultado != "":
            resultado += ", "
        resultado += str(c3["doc_identidad"])

    if resultado == "":
        resultado = "Ninguno"

    return resultado
```

Pregunta 4 (37 pts). Implemente la función `valor_matricula` en Python, que calcula el valor de la matrícula que le corresponde pagar a un candidato. Hay dos casos:

1. Si el puntaje del candidato para la beca del gobierno es igual a `1.0`, el valor de su matrícula es: `0.0`.
2. En caso contrario, el candidato paga la `matricula_plena` menos los descuentos a los que tenga derecho. Todos los descuentos se calculan sobre la `matricula_plena` y se acumulan entre sí.

2.1. Descuento por ingresos. Según los ingresos del núcleo familiar en salarios mínimos legales vigentes (SMLV), el candidato recibe uno solo de los siguientes descuentos:

Ingresos del núcleo familiar	Descuento
Menores o iguales a un SMLV	40 %
Mayores a un SMLV y menores o iguales a dos SMLV	20 %
Mayores a dos SMLV y menores o iguales a tres SMLV	10 %
Mayores a tres SMLV	0 %

2.2. Descuentos adicionales. Además del anterior, el candidato recibe un descuento del 10 % por cada una de estas tres condiciones que cumpla: 1) pertenecer a una comunidad indígena, 2) tener alguna discapacidad y 3) ser víctima del conflicto armado.

Por ejemplo, un candidato cuyos ingresos familiares son mayores a un SMLV y menores a dos SMLV recibe un 20 %. Si además es víctima del conflicto armado y tiene una discapacidad, acumula $20\% + 10\% + 10\% = 40\%$ de descuento acumulado, es decir, paga el 60 % de la `matricula_plena`.

- Debe crear **una sola aserción** que verifique que `candidato` sea del tipo `dict`, con su mensaje de error.
- **Debe** usar las constantes dadas, no valores literales, y redondear el resultado a los decimales que indica la documentación. **No** puede usar `len()`, `min()` ni `max()`; **sí** puede usar `round()`.

```

def valor_matricula(candidato: dict, matricula_plena: float, max_ingreso_nucleo_familiar: float) -> float:
    """
    Calcula el valor de la matrícula que le corresponde pagar a un candidato.

    Args:
        candidato (dict): Diccionario que representa al candidato.
        matricula_plena (float): Valor completo de la matrícula, sin ningún
            descuento. Ejemplo: 10000000.0.
        max_ingreso_nucleo_familiar (float): Ingreso máximo del núcleo familiar. Ejemplo: 2000000.0.

    Returns:
        float: Valor de la matrícula que debe pagar el candidato, redondeado a 2 decimales.
    """
    assert isinstance(candidato, dict), "Se esperaba un candidato del tipo dict, pero se recibió: " +
        str(type(candidato))

    UN_SMLV = 1750905.0
    DOS_SMLV = 3501810.0
    TRES_SMLV = 5252715.0
    DESCUENTO_HASTA_UN_SMLV = 0.4
    DESCUENTO_HASTA_DOS_SMLV = 0.2
    DESCUENTO_HASTA_TRES_SMLV = 0.1
    DESCUENTO_POR_CONDICION = 0.1
    SIN_DESCUENTO = 0.0
    PUNTAJE_BECA_COMPLETA = 1.0
    MATRICULA_BECA_COMPLETA = 0.0

    ingresos = candidato["ingresos_nucleo_familiar"]
    valor = MATRICULA_BECA_COMPLETA

    if puntaje_para_beca_gobierno(candidato, max_ingreso_nucleo_familiar) != PUNTAJE_BECA_COMPLETA:
        descuento = SIN_DESCUENTO

        if ingresos <= UN_SMLV:
            descuento = DESCUENTO_HASTA_UN_SMLV
        elif ingresos <= DOS_SMLV:
            descuento = DESCUENTO_HASTA_DOS_SMLV
        elif ingresos <= TRES_SMLV:
            descuento = DESCUENTO_HASTA_TRES_SMLV

        if candidato["es_indigena"]:
            descuento += DESCUENTO_POR_CONDICION
        if candidato["tiene_discapacidad"]:
            descuento += DESCUENTO_POR_CONDICION
        if candidato["es_victima_conflicto"]:
            descuento += DESCUENTO_POR_CONDICION

        valor = matricula_plena - (matricula_plena * descuento)

    return round(valor, 2)

```